codeHive

Logarithmic Search

THE BINARY ADVANTAGE

DSA Advanced Series: Vol 15

1. The Power of Halving

Binary Search is an extremely efficient algorithm for finding target values within a dataset. Unlike Linear Search, which checks every item, Binary Search eliminates **half** of the remaining search area with every single step.

The Golden Rule

For Binary Search to work, the array **MUST already be sorted**. If the data is scrambled, the logic of "eliminating halves" fails completely.

How it Works:

1. Find the value in the **center** of the array.
2. If center == target: Return the index.
3. If center > target: Ignore the right half; repeat search on the left.
4. If center < target: Ignore the left half; repeat search on the right.

2. Visualizing the Hunt

Target Value: **11**

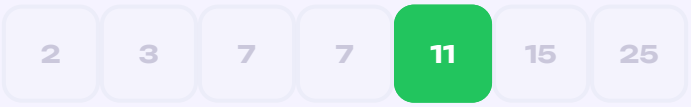
Step 1: Start Middle
Check index 3 (Value 7). 7 < 11, so we discard everything to the left.

Step 2: New Sub-Array

New search area is [11, 15, 25]. Check middle index 5 (Value 15). 15 > 11, so discard Right.

Step 3: Final Pivot

Only index 4 remains. Value is 11. Match found!



3. Python Implementation

This implementation uses a `while` loop to adjust the `left` and `right` pointers until they cross or the value is found.

```
def binarySearch(arr, targetVal):
    left = 0
    right = len(arr) - 1

    while left ⩽ right:
        # Calculate the middle index
        mid = (left + right) // 2

        if arr[mid] == targetVal:
            return mid

        # Adjust pointers to halve the area
        if arr[mid] < targetVal:
            left = mid + 1
        else:
            right = mid - 1

    return -1 # Value not found

# Execution
data = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
target = 15
print(f"Index: {binarySearch(data, target)}")
```

4. Big O Efficiency SCALABLE LOGIC

TIME COMPLEXITY

O(log n)

The logarithmic curve grows very slowly.

Why log(n) matters:

If you have an array of **1,000,000** elements:

- **Linear Search:** May take 1,000,000 comparisons.
- **Binary Search:** Takes at most **20** comparisons.

Even if the array size doubles to 2,000,000, Binary Search only needs **one extra check** (21 total).

5. Knowledge Check

Exercise: For an array of 200 elements, approximately how many comparisons are needed in the worst-case scenario for Binary Search?

Answer: ~8 comparisons (2^8 = 256).